Appendix A: Soft Code Flowcharts

The flowcharts below follow the project's actual simulation and control code; the source files are named in each caption, and all numerical values come from the project configuration (config/simulation_config.py). Standard symbols are used: rounded boxes for start and end, rectangles for processes, parallelograms for input and output, and diamonds for decisions.

The project runs in stages. The tank, pivot and motor are first built from simple shapes, with mass and inertia computed analytically. A PPO controller is then trained on the fast surrogate model (500,000 steps), evaluated over 20 episodes, and fine-tuned for 50,000 steps on rigid-body Chrono physics. A tank-angle trajectory produced by the trained controller is recorded and replayed in the SPH solver at four particle spacings, and the SPH results are compared with the pendulum and hydrostatic models. Finally, the controller is fine-tuned and tested in the two-way coupled SPH tank, where the liquid acts back on the tank. The Chrono::FSI steps ran on the two GPU workstations.

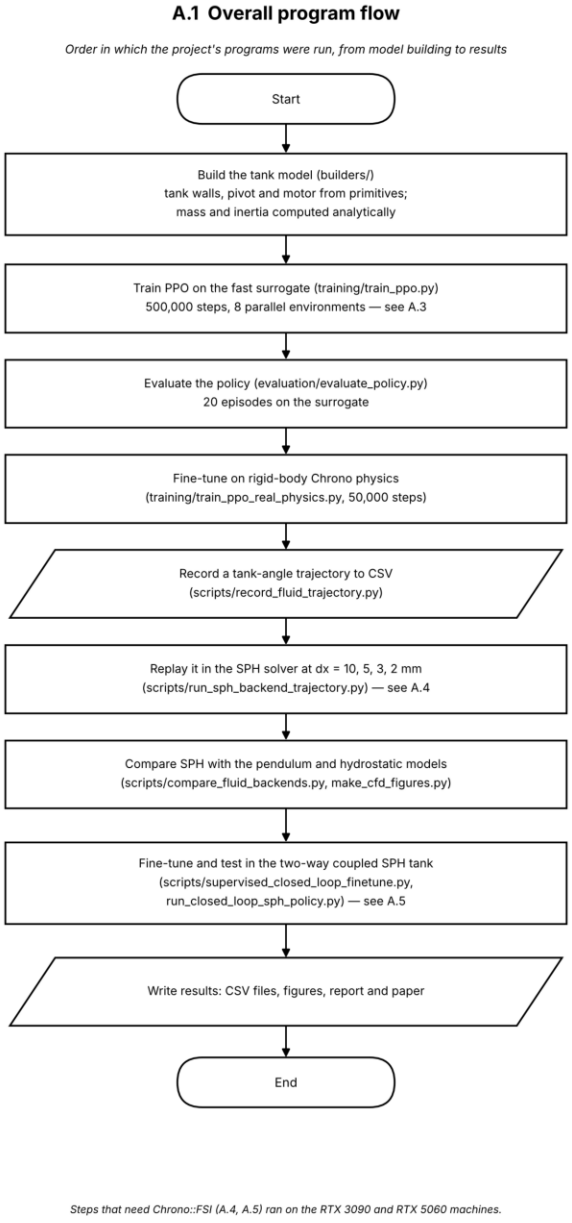


Figure A.1. Overall program flow: the order in which the project's programs were run.

Each episode begins with reset(): the tank and fluid model return to rest and the liquid receives a random impulse of 0–0.02 N·s in a random direction. Every 20 ms the controller reads a seven-value observation (tank angle and rate, liquid centre-of-mass position and velocity, and slosh energy) and returns an action between −1 and 1, which is clipped and scaled to a motor torque of at most 2 N·m. The torque is held for 20 physics substeps of 1 ms; in each substep the tank is advanced first and the fluid model is then advanced with the new tank angle and rate. After the substeps the reward is computed, penalising slosh energy, torque and tilt. The episode ends early if the tank tilts beyond 45°, and otherwise after 500 control steps (10 s). The same loop is used for PPO training, for evaluation and for the PID and LQR baselines.

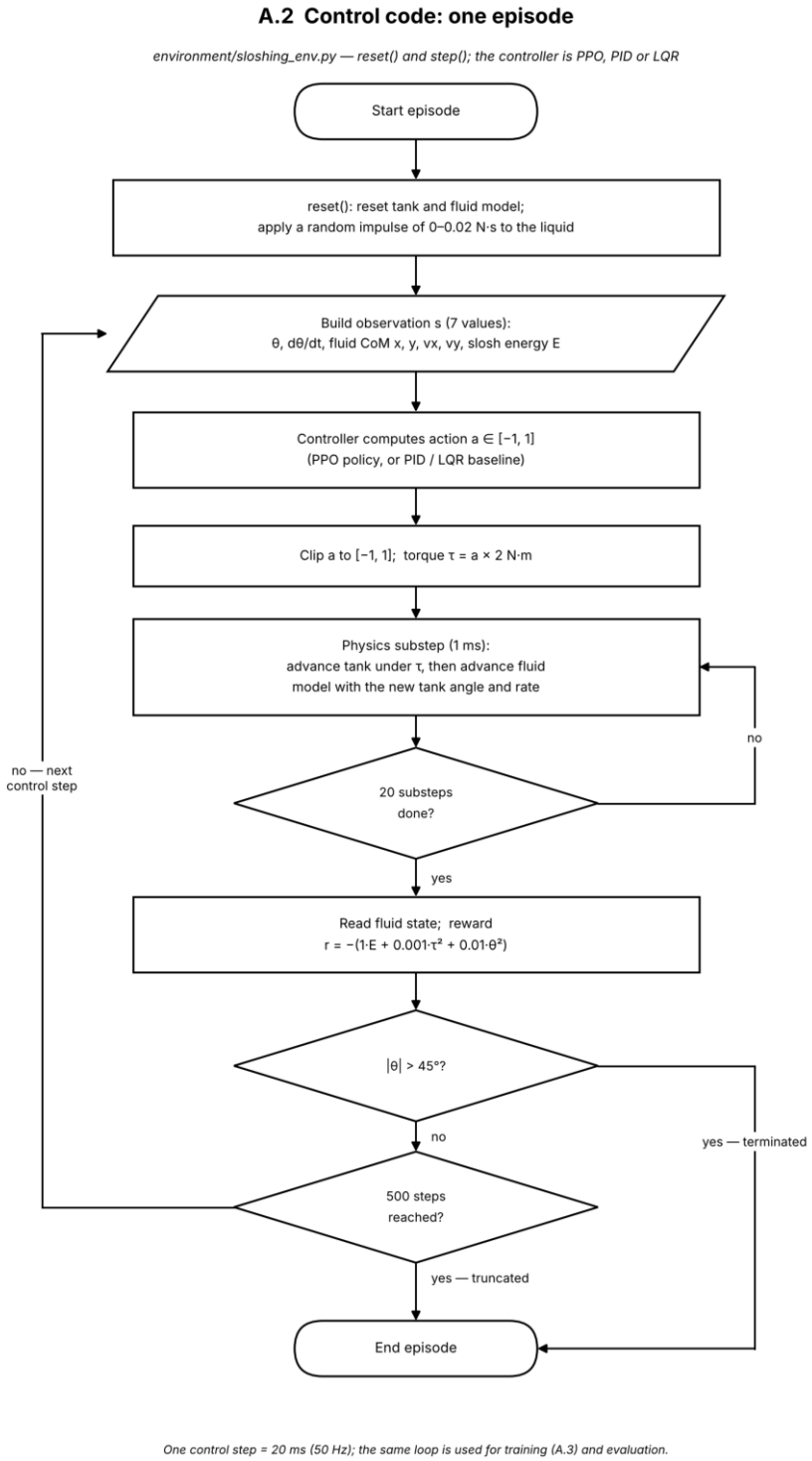


Figure A.2. Control code: one episode of the sloshing environment (environment/sloshing_env.py).

Training uses the stable-baselines3 implementation of PPO. Separate actor and critic networks, each with two hidden layers of 64 tanh units, are created, and eight copies of the environment of Fig. A.2 run in parallel behind a wrapper that normalises observations and rewards. Each round collects 1,024 steps from every environment (8,192 transitions), estimates advantages with GAE ($\gamma = 0.99$, $\lambda = 0.95$), and updates both networks for 10 epochs in minibatches of 256 using the clipped PPO objective ($\varepsilon = 0.2$). A checkpoint is saved every 50,000 steps, and after 500,000 steps (about 62 rounds) the final policy and its normalisation statistics are saved.

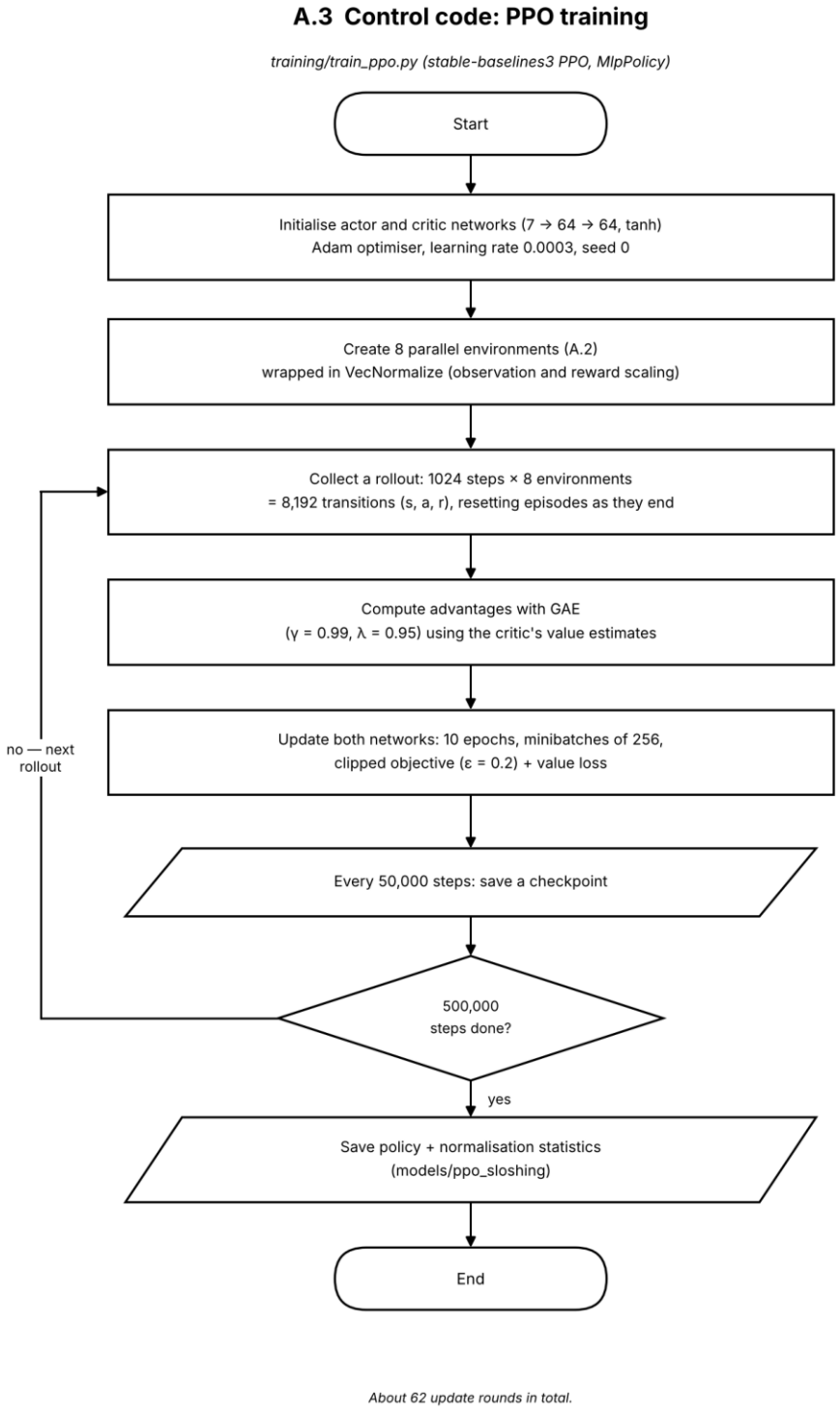


Figure A.3. Control code: PPO training (training/train_ppo.py).

In these runs the tank motion is prescribed rather than solved for (one-way coupling). The program reads the recorded trajectory (tank angle and rate every 20 ms) and first checks whether a complete output file already exists, in which case the run is skipped. Otherwise it builds the Chrono::FSI scene: the tank walls are converted to boundary markers and the water to a lattice of particles at spacing dx, with the solver settings described in the methodology. For each recorded row, the 20 ms interval is divided into solver substeps; in each one the tank angle is moved by linear interpolation towards the target and the SPH solver advances by one substep. At the end of the interval the liquid centroid, its velocity and the bulk kinetic energy are computed from the particles and stored. Results are written to disk every 25 rows, so an interruption loses little work.

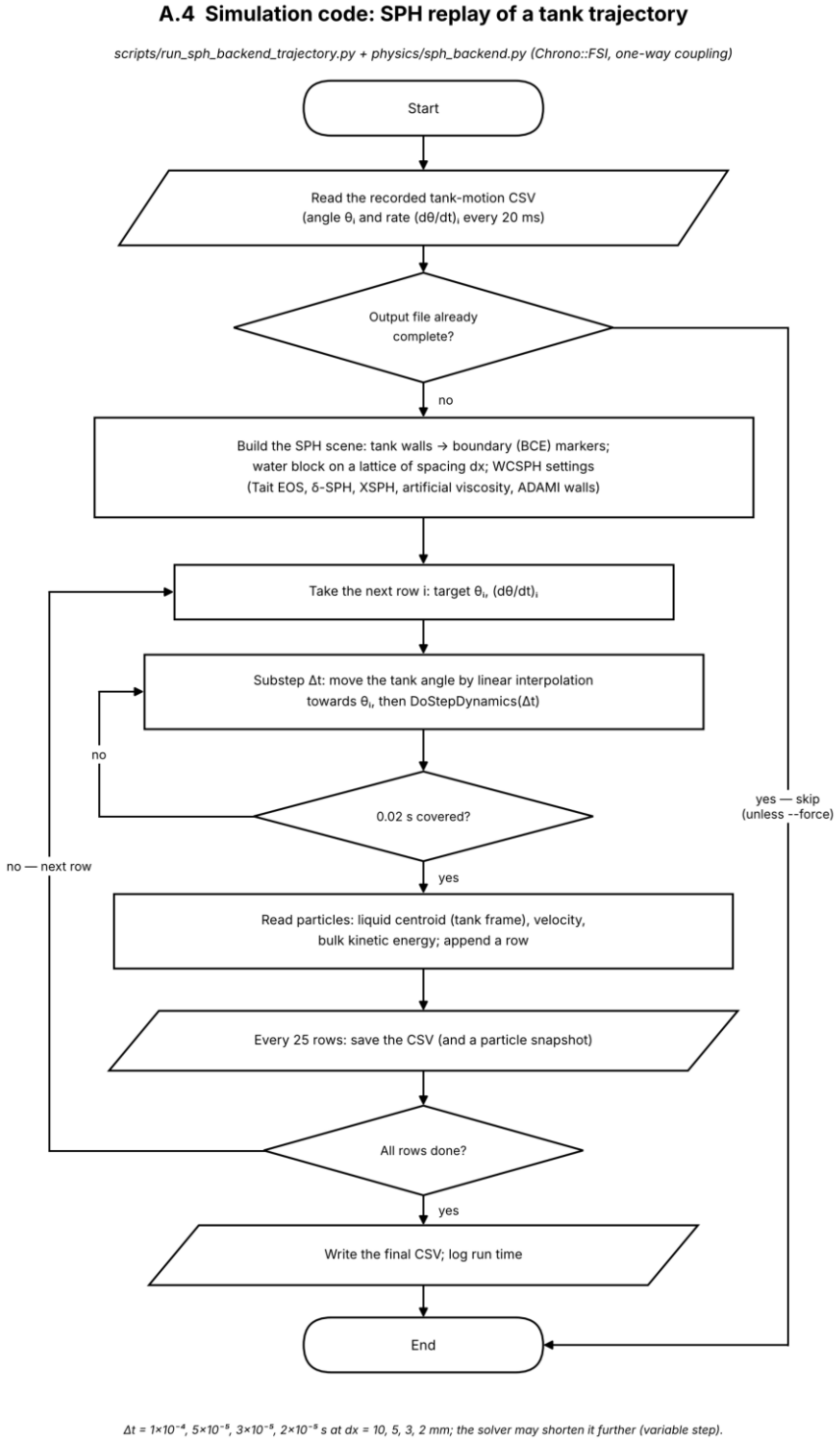


Figure A.4. Simulation code: SPH replay of a recorded tank trajectory (scripts/run_sph_backend_trajectory.py, physics/sph_backend.py).

Here the tank is driven by torque and the SPH liquid forces act back on it (two-way coupling). After loading the trained policy and its normalisation statistics, the program builds the tank with a torque motor at the pivot and starts it with a 0.05 rad/s kick. At each 20 ms step it first checks the tank's angular speed and stops the run if it exceeds 1.0 rad/s. Otherwise the policy chooses an action, which is converted to a torque limited to 7 N·m, and the coupled system advances in small substeps. The same speed check is repeated after every substep, so the run stops before the solver diverges rather than after. The tank state, the torque and the liquid quantities are recorded every step and saved periodically and at the end.

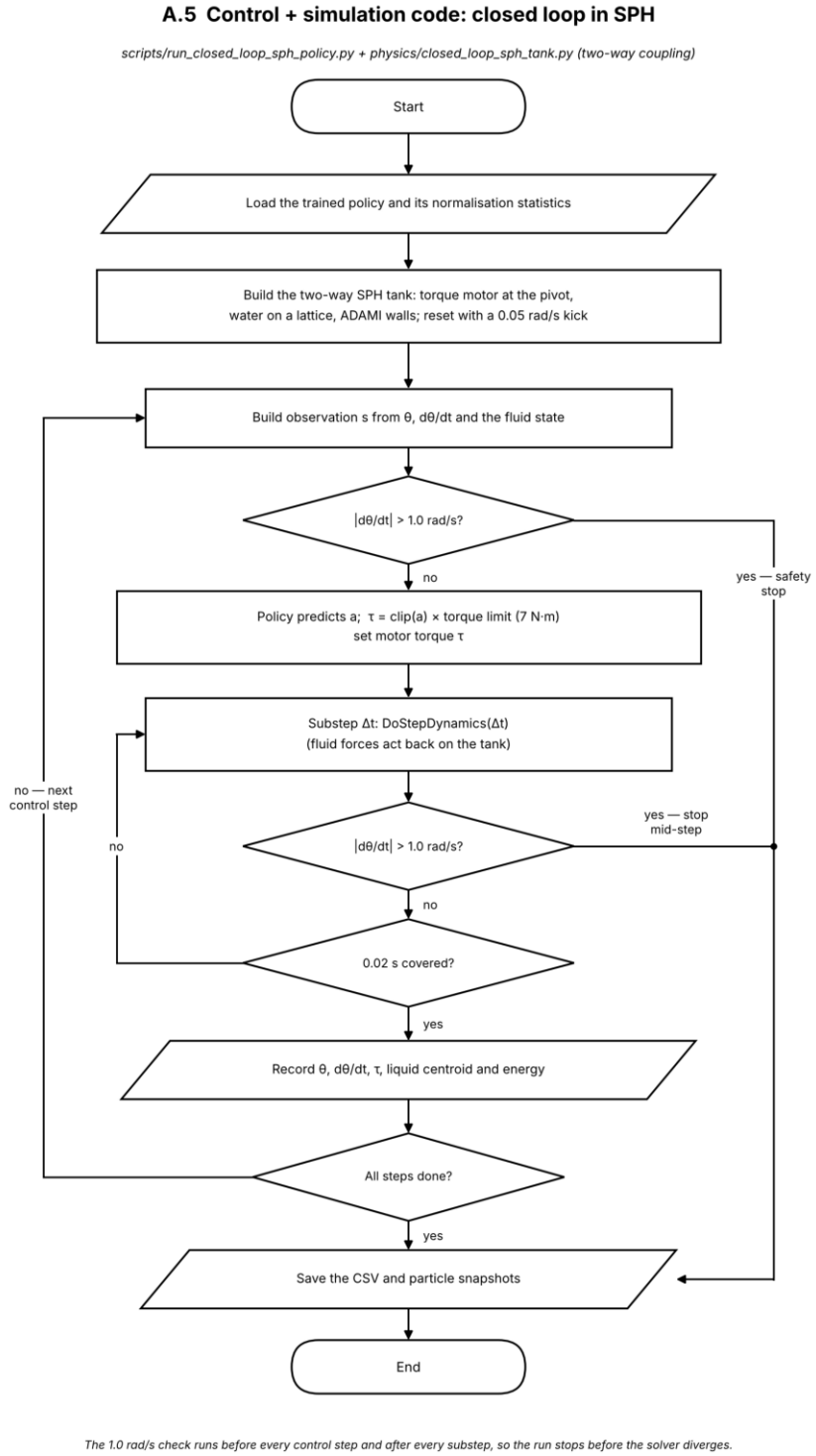


Figure A.5. Control and simulation code: closed loop in the two-way coupled SPH tank (scripts/run_closed_loop_sph_policy.py, physics/closed_loop_sph_tank.py).

The block diagram shows the same loop at system level. The goal, a level tank with no sloshing, reaches the controller through the reward during training. The controller output passes through the torque limit (±2 N·m in training, ±7 N·m in the SPH tank) to the motor at pivot O, which drives the plant: the tank on its pivot and the sloshing liquid, coupled in both directions. Disturbances act on the liquid. The measured tank angle, rate and liquid state are normalised and fed back as the seven-value state, closing the loop every 20 ms. The dashed path marks the training-only reward signal that PPO uses to update the controller.

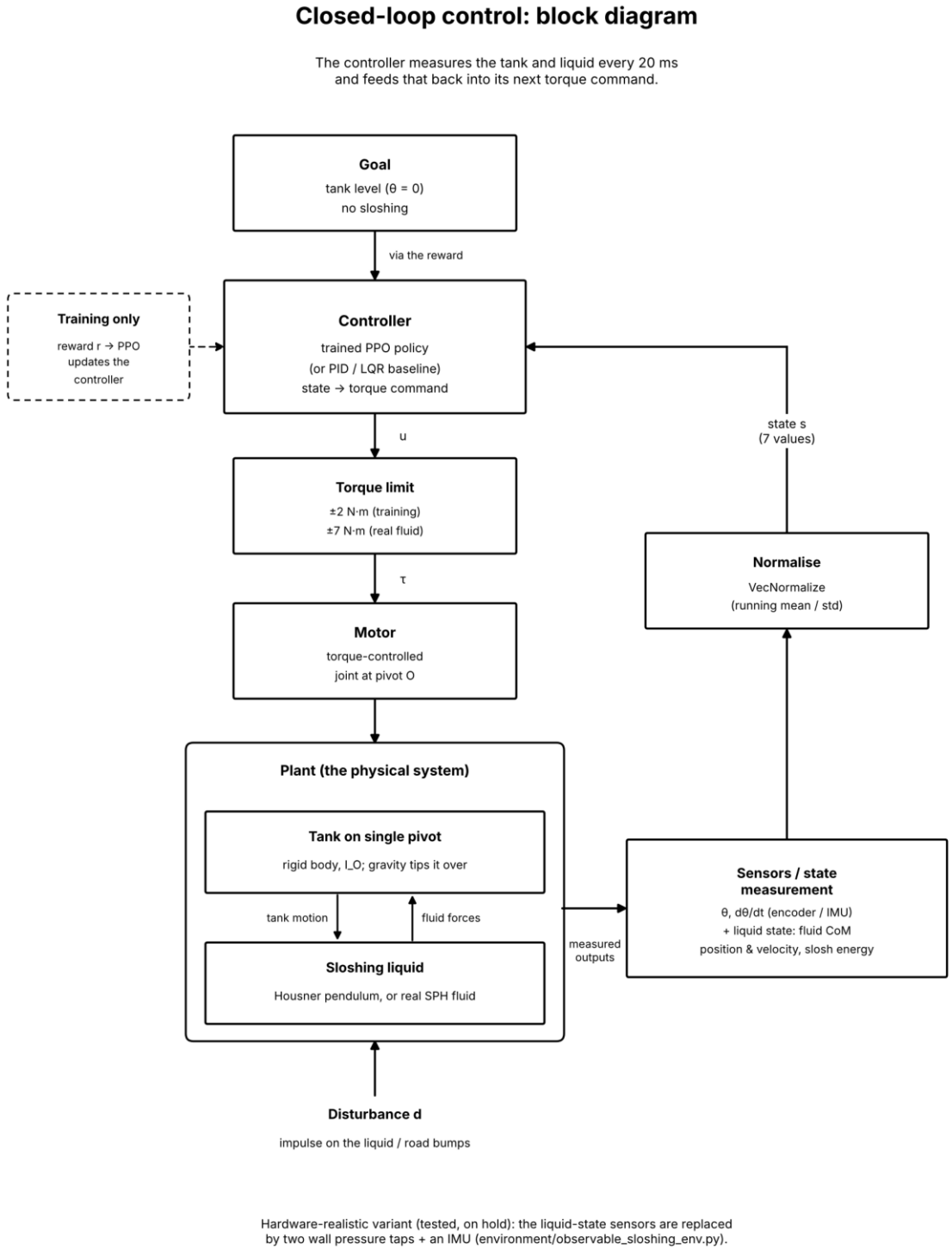


Figure A.6. Closed-loop control block diagram (system overview).